user@ossp:~$ ./start_presentation.sh --topic linux-debugging

Linux Debugging

Tools & Techniques

A practical guide to diagnosing and resolving system & application issues on Linux.

user@ossp:~$ _

OSSP PRESENTATION

OVERVIEW

What We'll Cover



Debugging Fundamentals

The mindset and process behind finding root causes.



GDB: Source-Level Debugging

Stepping through code and inspecting state.



Common Bug Categories

The usual suspects behind Linux system failures.



Tracing & Memory Tools

strace, ltrace, and Valgrind in action.



The Debugging Toolbox

Matching tools to the layer where a bug lives.



Logs, Performance & Best Practices

Observability tools and habits that save time.

CONCEPT

What Is Debugging?

Debugging is the systematic process of locating, understanding, and resolving defects that cause software or systems to behave unexpectedly. On Linux, that often means tracing a problem from application code down through libraries, system calls, and kernel behavior.

- Be systematic — follow a repeatable process, not guesswork
- Question assumptions — verify what you think is true
- Reproduce before you fix — an unreliable repro wastes time

1. **Reproduce**
   Recreate the issue reliably, every time
2. **Isolate**
   Narrow down the affected component
3. **Hypothesize**
   Form a theory about the root cause
4. **Test**
   Validate or reject it with the right tool
5. **Fix & Verify**
   Apply the fix and confirm resolution

PROBLEMS

Common Bug Categories in Linux Systems



Segmentation Faults

Invalid memory access causes the OS to terminate the process (SIGSEGV).



Memory Leaks

Allocated memory is never freed, gradually exhausting available RAM.



Race Conditions & Deadlocks

Concurrent threads interact unpredictably or wait on each other forever.



Failed System Calls

I/O, network, or resource calls return errors that go unchecked.



Resource Exhaustion

CPU, memory, or file-descriptor limits are hit under real load.



Permission & Config Errors

Incorrect file permissions or environment setup block execution.

The Linux Debugging Toolbox

Matching the right tool to the right layer of the problem



Source-Level

`gdb`



Tracing

`strace, ltrace`



Memory

`valgrind`



Logs

`dmesg, journalctl`



Performance

`top, htop, perf`

From source code to kernel — pick the tool that matches where the bug lives.

GDB — The GNU Debugger

GDB lets you pause a running program, step through code line by line, and inspect variables and memory — ideal for crashes, logic errors, and unexpected state.

Command	Purpose
`break <loc>`	Set a breakpoint at a line or function
`run`	Start the program under GDB
`next / step`	Execute line-by-line (step enters calls)
`print <var>`	Inspect a variable's current value
`backtrace`	Show the current call stack
`continue`	Resume until the next breakpoint

```
$ gdb ./myapp
(gdb) break main
Breakpoint 1 at 0x4005a6: file app.c, line 12.
(gdb) run
Starting program: /home/user/myapp

Breakpoint 1, main () at app.c:12
(gdb) print count
$1 = 0
(gdb) backtrace
#0  main () at app.c:12
(gdb) _
```

Tracing System & Library Calls



strace

Traces the system calls exchanged between your program and the Linux kernel.

```
strace -f ./app
```

USE WHEN

Diagnosing failed file access, network calls, or permission errors.



ltrace

Traces the calls your program makes into shared and dynamic libraries (e.g. libc).

```
ltrace ./app
```

USE WHEN

Diagnosing unexpected library behavior or missing symbols.

Together, strace and ltrace show what a program is really doing beneath the source code.

Finding Memory Bugs with Valgrind

Valgrind runs a program on a virtual CPU, checking every memory access for correctness — no recompilation required.

WHAT IT CATCHES

- Memory leaks — allocated blocks that are never freed
- Invalid reads / writes — buffer overruns and underruns
- Use-after-free — accessing memory already released
- Uninitialized memory — reading values before they are set

```
$ valgrind --leak-check=full ./app
==12345== HEAP SUMMARY:
==12345==     in use at exit: 40 bytes in 1 blocks
==12345==   total heap usage: 2 allocs, 1 frees

==12345== 40 bytes in 1 blocks are definitely lost
==12345==    at 0x4C2FB0E: malloc (vg_replace.c:299)
==12345==    by 0x4006B2: main (app.c:8)

==12345== LEAK SUMMARY:
==12345==    definitely lost: 40 bytes in 1 blocks
$ _
```

RUNTIME OBSERVABILITY

Logs & Performance Monitoring

Logs



dmesg

Prints the kernel ring buffer — hardware, driver, and boot messages.



journalctl / var/log

Centralized system and service logs managed by systemd.



/proc filesystem

A live, virtual window into kernel data and running processes.

Performance



top / htop

Real-time view of CPU, memory, and per-process resource use.



perf

Kernel-level profiler for CPU counters and call-graph hotspots.



vmstat / iostat

Snapshots of system-wide memory, CPU, and disk I/O activity.

Best Practices & Summary

The workflow, and the habits that make it faster

1 **Reproduce** ▶ 2 **Isolate** ▶ 3 **Hypothesize** ▶ 4 **Test** ▶ 5 **Fix**

Iterate until the issue is fully resolved

Key Takeaways

- Read error messages & logs first — the answer is often already there.
- Change one variable at a time to isolate cause and effect.
- Use version control to bisect regressions quickly.
- Automate reproduction — a flaky repro wastes hours.

Thank You